

Guía de Análisis: Aplicación VR con Three.js

— Drag, Shaders y Depth Sensing

Contexto: Este código es una aplicación de realidad virtual diseñada para funcionar en **teléfono móvil** usando WebXR. A diferencia de experiencias VR tradicionales, **no usa estereoscopía** (no se ve con ambos ojos simultáneamente), lo que significa que funciona en modo "pantalla única" o monocular.

1. ¿Qué hace esta aplicación?

Esta app te permite:

- **Ver un cubo 3D** frente a ti en el espacio virtual.
- **Agarrar y mover el cubo** usando los controladores VR (o la pantalla táctil del teléfono).
- **Detectar la profundidad real del entorno** (depth sensing) para que objetos virtuales se oculten detrás de objetos reales.

2. Estructura General del Código

El programa sigue el patrón clásico de Three.js:

INICIALIZACIÓN → CONFIGURACIÓN DE ESCENA → OBJETOS → CONTROLADORES → BUCLE DE ANIMACIÓN

Función	Propósito
<code>init()</code>	Crea toda la escena, cámara, luces, objetos y controladores
<code>animate() / render()</code>	Bucle infinito que actualiza cada fotograma
<code>onSelectStart()</code>	Se activa cuando presionas el botón del controlador (agarrar)
<code>onSelectEnd()</code>	Se activa cuando sueltas el botón (soltar)
<code>getIntersections()</code>	Calcula si el rayo del controlador toca un objeto

3. Configuración de la Escena

Cámara y Controles

```
camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 10);
camera.position.set(0, 1.6, 3);
```

```
controls = new OrbitControls(camera, container);
controls.target.set(0, 1.6, 0);
```

PerspectiveCamera(50, ...): Campo de visión de 50°. Los parámetros 0.1 y 10 definen el **near** y **far** — solo se renderizan objetos entre 10 cm y 10 metros.

● **camera.position.set(0, 1.6, 3):** La cámara está a 1.6m de altura (altura de ojos) y a 3 metros de profundidad.

● **OrbitControls:** Permite rotar la vista con el mouse o táctil cuando NO estás en modo VR.

Iluminación

```
scene.add(new THREE.HemisphereLight(0xbcbcbc, 0xa5a5a5, 3));
```

```
const light = new THREE.DirectionalLight(0xffffff, 3);
```

HemisphereLight: Luz ambiental que simula el cielo y el suelo.

● **DirectionalLight:** Luz direccional que proyecta **sombras** (`castShadow = true`). La sombra usa un mapa de 4096×4096 píxeles para alta calidad.

4. El Cubo: Posición y Material Shader

Posición del Cubo (¡Modificación importante!)

```
const object = new THREE.Mesh(geometry, material);
```

```
object.position.set(0, 1.0, -1.5);
```

Nota clave: El cubo se coloca en $z = -1.5$, es decir, **1.5 metros frente al usuario**. Esto es fundamental para que sea visible y alcanzable en VR.

El Material: ShaderMaterial

```
const material = new THREE.ShaderMaterial({
```

```
  vertexShader: `...`,
```

```
  fragmentShader: `...`
```

```
});
```

En lugar de usar materiales estándar como `MeshStandardMaterial`, aquí usamos **ShaderMaterial**. Esto significa que **tú escribes el shader** (programa GPU) que determina cómo se ve el objeto.

Uniforms (variables que pasamos del JavaScript al shader):

Uniform	Valor inicial	Función
diffuseColor	0x00aaff (azul)	Color base del cubo
emissive	0.0	Brillo de emisión (se enciende al agarrar)
roughness	1.0	Rugosidad superficial
metalness	0.0	No es metálico
depthColor	null	Textura de profundidad de la cámara
depthWidth / depthHeight	1	Dimensiones de la textura de profundidad

. 🎨 Análisis del Shader

Vertex Shader (Procesa cada vértice)

```
varying vec3 vNormal;
```

```
varying vec2 vUv;
```

```
void main() {  
    vNormal = normalize(normalMatrix * normal);  
    vUv = uv;  
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);  
}
```

Línea	Explicación
<code>vNormal normalize(normalMatrix normal)</code>	=Transforma la normal del objeto al espacio de la cámara y la normaliza. La * normal es el vector perpendicular a la superficie; se necesita para calcular la iluminación.
<code>vUv = uv</code>	Pasa las coordenadas de textura al fragment shader.
<code>gl_Position = ...</code>	Calcula la posición final del vértice en pantalla usando las matrices de proyección y modelo-vista.

varying: Son variables que "viajan" del vertex shader al fragment shader. Three.js interpola automáticamente sus valores entre vértices.

Fragment Shader (Procesa cada píxel)

```
uniform vec3 diffuseColor;

uniform float emissive;

varying vec3 vNormal;

void main() {
    vec3 lightDir = normalize(vec3(1.0, 1.0, 1.0));
    float lightIntensity = max(dot(vNormal, lightDir), 0.3);
    vec3 finalColor = diffuseColor * lightIntensity;
    finalColor += vec3(emissive * 0.2);
    gl_FragColor = vec4(finalColor, 1.0);
}
```

Línea	Explicación
<code>lightDir = normalize(vec3(1.0, 1.0, 1.0))</code>	Define una luz que viene desde la esquina superior derecha (vector diagonal).
<code>dot(vNormal, lightDir)</code>	Producto punto entre la normal y la dirección de la luz. Si la superficie mira hacia la luz, el valor es alto; si no, es bajo.
<code>max(..., 0.3)</code>	Garantiza que haya al menos un 30% de luz (iluminación ambiental mínima).
<code>finalColor += vec3(emissive * 0.2)</code>	Añade brillo emisivo. Cuando <code>emissive = 1.0</code> (al agarrar), el cubo se ilumina un 20% más.
<code>gl_FragColor = vec4(finalColor, 1.0)</code>	Color final del píxel con opacidad total (alfa = 1.0).

6. 🖐️ Funciones de Profundidad en el Shader

Aunque el fragment shader actual **no usa activamente** las funciones de profundidad (las calcula pero no las aplica al color final), están preparadas para la **oclusión de profundidad**:

Depth_GetCameraDepthInMeters

```
float Depth_GetCameraDepthInMeters(const sampler2DArray depthTexture, const vec2
depthUv, int arrayIndex) {
    return texture(depthColor, vec3(depthUv.x, depthUv.y, arrayIndex)).r;
}
```

Lee el valor de profundidad (en metros) de la textura de la cámara en una coordenada UV específica.

Depth_GetOcclusion

```
float Depth_GetOcclusion(...) {
    float depthMm = Depth_GetCameraDepthInMeters(...);
    return clamp(1.0 - 0.5 * (depthMm - assetDepthM) / (kDepthTolerancePerM * assetDepthM) + 0.5,
0.0, 1.0);
}
```

Compara la profundidad del objeto virtual (`assetDepthM`) con la profundidad real del mundo (`depthMm`). Si el mundo está más cerca, devuelve un valor cercano a 0 (oculto); si el objeto está más cerca, devuelve 1 (visible).

Depth_GetBlurredOcclusionAroundUV

Aplica un **kernel de desenfoque gaussiano** sobre 25 muestras alrededor del píxel. Esto suaviza los bordes de la oclusión para que no se vea "pixelado".

Peso total del kernel: 269 — cada muestra se multiplica por un peso diferente (4, 7, 16, 26, 33) según su distancia al centro.

7. 🎮 Sistema de Drag (Arrastre)

Agarrar un objeto (onSelectStart)

```
function onSelectStart(event) {  
    const controller = event.target;  
    const intersections = getIntersections(controller);  
  
    if (intersections.length > 0) {  
        const object = intersections[0].object;  
        object.material.uniforms.emissive.value = 1.0; // Se ilumina  
        controller.attach(object);                // El controlador "adopta" el objeto  
        controller.userData.selected = object;      // Guarda referencia  
    }  
}
```

controller.attach(object): Es la magia del drag. El objeto deja de ser hijo del group y pasa a ser hijo del controlador. Como el controlador se mueve con tu mano, **el objeto se mueve con él.**

Soltar un objeto (onSelectEnd)

```
function onSelectEnd(event) {  
    const object = controller.userData.selected;  
    object.material.uniforms.emissive.value = 0.0; // Deja de brillar  
    group.attach(object);                // Vuelve al grupo original  
    object.rotation.set(0, 0, 0);        // ¡Resetea rotación!  
    controller.userData.selected = undefined;  
}
```

group.attach(object): El objeto vuelve a ser hijo del grupo, pero mantiene su posición y orientación en el mundo. **object.rotation.set(0, 0, 0): Modificación clave** — al soltar, el cubo se alinea perfectamente (sin rotación). Esto evita que quede torcido.

8. La Profundidad (Depth Sensing) — ¡El Corazón del Proyecto!

¿Qué es el Depth Sensing?

Es una tecnología que usa la **cámara de profundidad** del dispositivo (como LiDAR en iPhone o sensores ToF en Android) para crear un **mapa de profundidad** del mundo real. Cada píxel de ese mapa indica **qué tan lejos está el objeto real** de la cámara.

Configuración en WebXR

```
document.body.appendChild(XRButton.createButton(renderer, {  
  'optionalFeatures': ['depth-sensing'],  
  'depthSensing': {  
    'usagePreference': ['gpu-optimized'],  
    'dataFormatPreference': ['luminance-alpha']  
  }  
}));
```

Parámetro	Significado
<code>optionalFeatures: ['depth-sensing']</code>	Solicita acceso al sensor de profundidad del dispositivo
<code>usagePreference: ['gpu-optimized']</code>	Pide que los datos se procesen en la GPU (más rápido)
<code>dataFormatPreference: ['luminance-alpha']</code>	Formato de 2 canales: luminancia (profundidad) + alfa

Aplicación de la Profundidad en el Bucle de Render

```
if (renderer.xr.hasDepthSensing() && !isDepthSupplied) {  
  group.children.forEach(child => {  
    const depthTexture = renderer.xr.getDepthTexture();  
    child.material.uniforms.depthColor.value = depthTexture;  
  
    if (depthTexture && depthTexture.source && depthTexture.source.data) {  
      child.material.uniforms.depthWidth.value = depthTexture.source.data.width;  
      child.material.uniforms.depthHeight.value = depthTexture.source.data.height;
```

```

    } else {
        child.material.uniforms.depthWidth.value = 1680;
        child.material.uniforms.depthHeight.value = 1760;
    }
    isDepthSupplied = true;
});
}

```

Línea	Explicación
<code>renderer.xr.hasDepthSensing()</code>	Verifica si el dispositivo soporta y ha activado depth sensing
<code>renderer.xr.getDepthTexture()</code>	Obtiene la textura de profundidad como <code>sampler2DArray</code>
<code>depthWidth = 1680, depthHeight = 1760</code>	Valores por defecto si no se puede leer el tamaño real

¿Por qué es importante la profundidad?

Sin depth sensing: El cubo virtual siempre se dibuja "encima" de todo, incluso si en la realidad hay una mesa o una pared entre tú y el cubo. Se ve "falso".

Con depth sensing: El shader puede comparar la profundidad del cubo con la profundidad del mundo real. Si una mesa real está más cerca que el cubo, el cubo se **oculta parcialmente** detrás de ella. ¡Esto crea una mezcla perfecta entre realidad y virtualidad! 🌟

9. Modificaciones Específicas del Código

El código original fue modificado con estas líneas clave:

Modificación	Ubicación	Propósito
<code>object.position.set(0, 1.0, -1.5)</code>	Creación del cubo	Coloca el cubo a 1.5m de distancia, visible y alcanzable
<code>object.rotation.set(0, 0, 0)</code>	<code>onSelectEnd</code>	Resetea la rotación al soltar para mantener alineación
<code>group.rotation.set(0, 0, 0)</code>	<code>render()</code>	Bloquea la rotación del grupo para que la base siempre mire abajo

10. 📄 Resumen para Estudiantes

FLUJO DE LA APLICACIÓN
1. Inicializa escena, cámara, luces y renderizador 2. Crea un cubo con ShaderMaterial personalizado 3. Configura WebXR con depth sensing opcional 4. Crea 2 controladores VR con modelos y rayos visuales 5. En cada fotograma (render loop): • Si hay depth sensing → pasa textura al shader • Calcula intersecciones del rayo con el cubo • Si presionas botón → agarras el cubo (attach) • Si sueltas botón → sueltas el cubo (group.attach) • Dibuja la escena

Conceptos Clave a Recordar

Concepto	Definición Simple
Shader	Programa que corre en la GPU para dibujar píxeles
Uniform	Variable constante para todos los píxeles en un fotograma
Varying	Variable que se interpola entre vértices
Normal	Vector perpendicular a una superficie (para luz)
Depth Sensing	Tecnología que mide distancias del mundo real
Oclusión	Efecto de que un objeto se esconda detrás de otro
Attach/Detach	Cambiar el "padre" de un objeto en la jerarquía de escena

💡 **Consejo final:** Este código es un excelente punto de partida para entender cómo combinar **interacción 3D**, **shaders personalizados** y **realidad aumentada/mixta**. El depth sensing es lo que eleva una simple app 3D a una experiencia inmersiva creíble. ¡Experimenta modificando el color, la posición y el tamaño del kernel de oclusión!

Código QR

